

QMWS Spring 2026: Introduction to Python for Data Analysis

Day Two: Extensions

Gavin Klorfine & Deborah Laze

Section 1

Importing Packages, Defining Functions

Importing Packages

- Packages add “extra stuff” to Python, including functions, constants, and more
- Some packages are installed with Python by default, and no extra work is required to use them

```
import math
```

```
import math
```

```
print(f'pi = {math.pi:.3f}')
```

```
pi = 3.142
```

Importing Packages

- We can “rename” packages to make their usage more concise

“Renaming” Packages

```
import math as m

print(f'pi = {m.pi:.3f}')

pi = 3.142
```

Importing Packages

- Some require installation. This is most easily (and safely) done through **Anaconda**
- Let's install the package **NumPy**!

Importing Packages

```
import numpy as np
```

```
import numpy as np          # Common convention
```

```
# Take a sample of 3 values from N(0, 1)
```

```
print(np.random.normal(loc = 0, scale = 1, size = 3))
```

```
[ 0.65544581 -0.43829391  0.22284287]
```

Note

- $\text{loc} = \mu$, $\text{scale} = \sigma$

Importing Packages

- A brief aside:
 - Let's say of all the functions in **NumPy**, you only care about `numpy.random.normal`

```
from ... import ...
```

```
from numpy.random import normal
```

```
# Same thing, but no `np.random` at the start
```

```
normal(loc = 0, scale = 1, size = 3)
```

```
array([-1.38642688, -1.004365 ,  1.13824252])
```

Defining Functions

- Sometimes you repeat the same code over and over again
 - When this happens, it is useful to collapse the repeated code into a function
 - Otherwise, code becomes long and messy due to copy-pasting

Defining Functions

- Sometimes you repeat the same code over and over again
 - When this happens, it is useful to collapse the repeated code into a function
 - Otherwise, code becomes long and messy due to copy-pasting
 - For instance, imagine needing to know how to code your own `np.random.normal()` function and then copy pasting this code every time you want to use it

Defining Functions

Defining Basic Functions

```
def addition(a, b):  
    return a + b          # What the functions spits out  
  
print(addition(a = 1, b = 5)) # Call function  
  
def subtraction(a, b = 1):    # Give b default value of 1  
    return a - b  
  
print(subtraction(a = 5))
```

6

4

Defining Functions

Basic Structure of a Function

```
# "arg" short for "argument"  
def function_name(arg1, arg2 = default_value, arg_n):  
    return output
```

Defining Functions

Function Example

```
x = [0,1,2,3,4,5,6,7]
```

```
def qm_mean(things):
```

```
    sum = 0
```

```
    for i in things:
```

```
        sum += i
```

```
    return sum / len(things)
```

```
qm_mean(x)
```

```
3.5
```

Section 2

NumPy

- Somewhat similar to MATLAB if you have used that
- Useful for working with arrays/matrices
- Stands for **N**umerical **P**ython
- Based in computer language **C**
 - Faster, more efficient computations

Working With Lists/Arrays

```
import numpy as np
```

```
list1 = [1,2,3,4]  
print(list1 * 2)  
print(type(list1))
```

```
numpyList = np.array(list1) # Convert list -> numpy array  
print(numpyList * 2)  
print(type(numpyList))
```

```
[1, 2, 3, 4, 1, 2, 3, 4]  
<class 'list'>  
[2 4 6 8]  
<class 'numpy.ndarray'>
```

- Matrices are typically worked with as **2D** numpy arrays

2D NumPy Arrays

```
array2d = np.array([[1,2,3], [4,5,6]]) # List of lists
print(array2d)
print(type(array2d))
```

```
[[1 2 3]
```

```
 [4 5 6]]
```

```
<class 'numpy.ndarray'>
```


To Check Attributes of a NumPy Variable

```
array2d = np.array([[1,2,3], # Matrices typed in a more  
                    [4,5,6]]) # appealing manner
```

```
print(array2d.shape)      # Shape (# rows, # columns)  
print(array2d.ndim)      # Number of dimensions  
print(array2d.size)      # Number of elements  
print(array2d.dtype)     # Type of elements
```

```
(2, 3)
```

```
2
```

```
6
```

```
int64
```

- Some numpy types:
 - `uint8`
 - `int32`
 - `int64`
 - `float64`
- Generally, numpy type = type learned before + memory (in bits)

Note

- The `u` in `uint8` stands for “unsigned,” meaning that the variable can only store positive values

```
np.arange()
```

```
np.arange(start=10, stop=20, step=2, dtype = "int64")  
array([10, 12, 14, 16, 18])
```

Note

- stop parameter is *exclusive*

```
np.linspace()
```

```
np.linspace(start=10, stop=20, num=6, dtype = "int64")  
array([10, 12, 14, 16, 18, 20])
```

Tip

- Use `np.linspace()` for breaking something into equal parts
- Use `np.arange()` for creating a sequence of numbers

- NumPy has lots of functions relating to choosing random values

Random Choice

```
print(np.random.choice(["A", "B", "C", "D"]))
```

```
print(np.random.choice(["A", "B", "C", "D"], size = 2))
```

```
C
```

```
['C' 'C']
```

Random Numbers

```
print(np.random.normal(loc = 0, scale = 1, size = 3))  
print(np.random.uniform(low = 0, high = 10, size = 3))  
  
[ 1.07481607 -0.8694699   0.29564407]  
[3.13654554 7.19836152 5.0997414 ]
```

- To make the values from these random functions reproducible, we set a “seed”
 - i.e., the first time the below `np.random.choice()` call is run will always return 'D', the second time 'B'

Setting a Seed

```
np.random.seed(67)
```

```
# 1st time returns 'D'
```

```
print(np.random.choice(["A", "B", "C", "D"]))
```

```
# 2nd time returns 'B'
```

```
print(np.random.choice(["A", "B", "C", "D"]))
```

```
D
```

```
B
```

Indexing NumPy Arrays

```
x = np.random.normal(0, 10, size=(3,3))
print(x, "\n")  # Print x with a blank line after it

print(x[2,2], type(x[2,2]))
print(x[2,:])    # Row 2 (zero-based), all columns

[[ 4.24396269  8.19276386 -4.55523498]
 [-1.74348913 -12.91666321 -5.12083276]
 [ 1.51374927  2.9382983 -14.96950941]]

-14.96950940608669 <class 'numpy.float64'>
[ 1.51374927  2.9382983 -14.96950941]
```


Indexing (cont.)

```
x = np.random.normal(0, 10, size=(3,3))  
print(x, "\n")
```

```
print(x[2, 0:2])      # Row 2, columns 0 and 1  
print(type(x[2, 0:2]))
```

```
[[  5.54692736  -9.74439429  -5.77253404]  
 [-12.7560138   -7.86514849  -0.27373561]  
 [-1.48018218  -5.27834347  -1.01387863]]
```

```
[-1.48018218 -5.27834347]  
<class 'numpy.ndarray'>
```

- You may also perform **aggregate** functions on numpy arrays

NumPy Aggregate Functions

```
x = np.array([1,4,7,15])
```

```
print(np.mean(x))  
print(np.median(x))  
print(np.std(x))
```

6.75

5.5

5.2141634036535525

More Aggregate Functions

```
x = np.array([1,4,7,15])

print(np.min(x))           # Minimum value
print(np.argmin(x))        # Position of minimum value
print(np.max(x))           # Maximum value
print(np.argmax(x))        # Position of maximum value

1
0
15
3
```

Check-in

- How might you generate 100 random values from the standard normal distribution?
- How might you get the maximum of these values?

Check-in

- How might you generate 100 random values from the standard normal distribution?
- How might you get the maximum of these values?

```
np.max(np.random.normal(loc=0, scale=1, size=100))
```

```
np.float64(1.6230497183839991)
```

Section 3

pandas

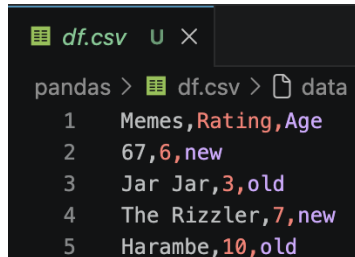
- For the analysis of spreadsheet-like data in Python
- Somewhat similar to **R** if you have used that
 - Introduces the DataFrame

Creating a DataFrame

```
import pandas as pd
df = pd.DataFrame({
    "Memes" : ["67", "Jar Jar", "The Rizzler", "Harambe"],
    "Rating": [6, 3, 7, 10],
    "Age"    : ["new", "old", "new", "old"]
}, index = ['a','b','c','d']) # Index is optional
df # To print
```

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old
c	The Rizzler	7	new
d	Harambe	10	old

- Data are commonly stored as .csv files
 - Comma separated values
- These files have entries separated by commas, often including a first row of “header” entries
- A .csv file of the **pandas** DataFrame from the previous slide (variable `df`) is shown below



```
df.csv U X
pandas > df.csv > data
1    Memes,Rating,Age
2    67,6,new
3    Jar Jar,3,old
4    The Rizzler,7,new
5    Harambe,10,old
```

- Continuing with the df DataFrame, the below shows how a .csv file is written

Writing .csv Files

```
# Write df.csv to the pandas/ folder  
df.to_csv("pandas/df.csv", index = False)
```

- Here, to_csv is a **method** acting on the df **object** (or variable)

Tip

Setting the index argument to False removes row indices (i.e., row 0, 1, 2, ..., n) from being stored as a separate column

- We often need to do the reverse; namely, reading data from a .csv file into a **pandas** DataFrame

Reading .csv Files

```
# Read df.csv from the pandas/ folder  
df_new = pd.read_csv("pandas/df.csv")
```

```
df_new
```

	Memes	Rating	Age
0	67	6	new
1	Jar Jar	3	old
2	The Rizzler	7	new
3	Harambe	10	old

- **pandas** also has functions to read/write data from other file formats, such as from Microsoft Excel `.xlsx` files

`.xlsx` Files

```
excelDat = pd.read_excel( # Read
    "file_name.xlsx", sheet_name = "optional"
)

excelDat.to_excel( # Write
    "folder/excelDat.xlsx", sheet_name = "optional"
)
```

Inspecting Data

```
df.head(2) # First 2 rows
```

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old

```
df.tail(2) # Last 2 rows
```

	Memes	Rating	Age
c	The Rizzler	7	new
d	Harambe	10	old

Inspecting Data

```
df.info()
```

```
<class 'pandas.DataFrame'>
```

```
Index: 4 entries, a to d
```

```
Data columns (total 3 columns):
```

#	Column	Non-Null Count	Dtype
0	Memes	4 non-null	str
1	Rating	4 non-null	int64
2	Age	4 non-null	str

```
dtypes: int64(1), str(2)
```

```
memory usage: 128.0+ bytes
```

Descriptives

```
df.describe()
```

	Rating
count	4.000000
mean	6.500000
std	2.886751
min	3.000000
25%	5.250000
50%	6.500000
75%	7.750000
max	10.000000

Grouping

```
df.groupby("Age").describe()
```

Age	Rating							
	count	mean	std	min	25%	50%	75%	max
new	2.0	6.5	0.707107	6.0	6.25	6.5	6.75	7.0
old	2.0	6.5	4.949747	3.0	4.75	6.5	8.25	10.0

Selecting a Column

```
df["Memes"]
```

```
a          67
```

```
b      Jar Jar
```

```
c  The Rizzler
```

```
d      Harambe
```

```
Name: Memes, dtype: str
```

A Note on Types

```
type(df["Memes"])
```

pandas.Series

- A variable of type pandas.Series is essentially a column of a **pandas DataFrame**

- To select multiple columns, provide a list of column names as the index

Selecting Multiple Columns

```
df[["Memes", "Age"]]
```

	Memes	Age
a	67	new
b	Jar Jar	old
c	The Rizzler	new
d	Harambe	old

- A simple way to filter rows is by row number:

Filter Rows by Row Number

```
df[0:2] # Row numbers 0 and 1
```

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old

- You can also filter rows by indexing a DataFrame with a boolean expression involving the same DataFrame

Filter Rows With Boolean Expressions

```
df[df["Age"] == "old"]
```

	Memes	Rating	Age
b	Jar Jar	3	old
d	Harambe	10	old

- The result of the previous slide was also a DataFrame!
 - This means we can index it further by, say, taking the "Memes" column:

Indexing Twice

```
df[df["Age"] == "old"]["Memes"]
```

```
b    Jar Jar
```

```
d    Harambe
```

```
Name: Memes, dtype: str
```

- This also means that methods shown previously can be applied to these filtered DataFrames

Describing Filtered Data

```
df[df["Age"] == "old"].describe()
```

	Rating
count	2.000000
mean	6.500000
std	4.949747
min	3.000000
25%	4.750000
50%	6.500000
75%	8.250000
max	10.000000

- We can also select rows and columns using the `loc` or `iloc` **attributes**
 - These are similar to methods but use square `[]` brackets (instead of round brackets `()`)

Note

A brief note on terminology: - The distinction between *methods* and *attributes* is that an attribute gives access to information associated with a variable; whereas a method is a function that may act on/with the variable. + E.g., `.loc` versus `.mean()`

Selecting with `loc` and `iloc`

```
df.loc["b", "Memes"] # Row b of Memes column
'Jar Jar'
df.iloc[1,0] # Row 1 of 0th column (Memes)
```


- Like **NumPy**, we can use a colon (:) with `loc` and `iloc` to indicate “all rows” or “all columns”:

Selecting All Rows with `loc`

```
df.loc[:, "Memes"] # All rows of Memes column
```

```
a          67
b      Jar Jar
c  The Rizzler
d      Harambe
Name: Memes, dtype: str
```

Selecting All Columns with `iloc`

```
df.iloc[0] # All columns for 0th row (row 'a')
```

```
Memes      67
```

```
Rating      6
```

```
Age         new
```

```
Name: a, dtype: object
```

i Note

- If only the row argument is supplied, all columns are selected by default
 - This is also true for `loc`

- If you would like to select multiple columns (or rows), but not all columns (or rows), pass a list to `loc` or `iloc`

Selecting Multiple (Specific) Columns with `loc`

```
df.loc[["a", "b"], ["Memes", "Age"]]
```

	Memes	Age
a	67	new
b	Jar Jar	old

- Same thing for `iloc`, but with row numbers instead of indices/names

- We can also use boolean expressions with `loc` (but not `iloc`):

Boolean Expressions with `loc`

```
# Take rows where age is old and just the meme column  
df.loc[df["Age"] == "old", "Memes"] # Old memes!
```

```
b    Jar Jar  
d    Harambe  
Name: Memes, dtype: str
```

- In these boolean expressions, we can use & for and, | for or, and ~ for not
 - We need these special operators as we are comparing entire columns as opposed to single values
 - We also surround expressions to be compared in round brackets ()

Boolean Expressions with loc

```
# Take all rows where age is old and rating is larger than 7
df.loc[(df["Age"] == "old") & (df["Rating"] > 7), :]
```

	Memes	Rating	Age
d	Harambe	10	old

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old
c	The Rizzler	7	new
d	Harambe	10	old

How might I index rows with rating greater than or equal to seven?

- I want 2 different answers! (Hint: `loc` and `df[?]`)

How might I index rows with rating greater than or equal to seven?

```
df[df["Rating"] >= 7]  
df.loc[df["Rating"] >= 7, :] # The `, :` is optional
```

	Memes	Rating	Age
c	The Rizzler	7	new
d	Harambe	10	old

- To show a peculiarity of pandas, I first set the index of our data frame df (e.g., the row names "a", "b", ...) back to integers (the default)

Changing the Index

```
df.index = [0,1,2,3]  
df
```

	Memes	Rating	Age
0	67	6	new
1	Jar Jar	3	old
2	The Rizzler	7	new
3	Harambe	10	old

- When using `loc` on a data frame with a numeric index, the format of a slice is `inclusive:inclusive`

loc is Strange

```
df.loc[0:1] # inclusive:inclusive
```

	Memes	Rating	Age
0	67	6	new
1	Jar Jar	3	old

- For reference, here is the same line but using `iloc` instead of `loc`:

`iloc` is Typical

```
df.iloc[0:1] # inclusive:exclusive
```

	Memes	Rating	Age
0	67	6	new

- We can also add columns to our data frame. Here I add a column r10: the Rating out of 10

Adding a Column

```
df["r10"] = df["Rating"] / 10  
df
```

	Memes	Rating	Age	r10
0	67	6	new	0.6
1	Jar Jar	3	old	0.3
2	The Rizzler	7	new	0.7
3	Harambe	10	old	1.0

- We might also want to sort these entries by Rating

Sorting a Data Frame

```
df.sort_values(by = "Rating", inplace = True)  
df
```

	Memes	Rating	Age	r10
1	Jar Jar	3	old	0.3
0	67	6	new	0.6
2	The Rizzler	7	new	0.7
3	Harambe	10	old	1.0

- Argument `inplace = True` modifies `df` directly instead of returning a new one

- We can also sort by Age first and then Rating
 - Strings are sorted alphabetically

Sorting by a List

```
df.sort_values(by = ["Age", "Rating"], inplace = True)  
df
```

	Memos	Rating	Age	r10
0	67	6	new	0.6
2	The Rizzler	7	new	0.7
1	Jar Jar	3	old	0.3
3	Harambe	10	old	1.0

- Instead of using the `describe()` method, you may want only a select few (or one) statistic

Getting Specific Descriptive Statistics

```
df["Rating"].mean()
```

```
np.float64(6.5)
```

```
df["Rating"].std()
```

```
np.float64(2.886751345948129)
```

Getting Specific Descriptive Statistics

```
df["Rating"].agg(['mean', 'std']) # agg = aggregate
```

```
mean    6.500000
```

```
std      2.886751
```

```
Name: Rating, dtype: float64
```